

CVPR 2026 DriveX Workshop
Tesla AI
Phil Duan
Self-Driving at Scale with Foundation Models

Today, if you buy a Tesla vehicle, this is the only vehicle you can buy here that can take you from point A to point B without any input. All you need to do is to put in the navigation address and then press a button. The car is capable of driving by itself everywhere and is capable of doing lane changes and turns and going through constructions and drives like a normal human. And all the input we have is just eight cameras surrounding the vehicle, and the local compute residing inside the vehicle. So we don't need any extra sensors or any external infrastructure to make this work.

The vehicle essentially is paying attention to everything around it at a very high frame rate. And it is able to make extremely responsive actions towards anything that happens on the road.

In order to take it one step further, we're also trying to build an unsupervised vehicle on the road. So currently, in Austin, Dallas, and Houston, we have dozens of these Robotaxi vehicles running with nobody in the driver's seat and nobody supervising. All you need to do is to call a car like we do with Uber and get in the car and the car will take you to the destination. And the car essentially uses exactly the same computer and sensors as any Model Y you can buy on the market. So we're also scaling up the fleet in both geolocations and also the quantity of cars. So in the next few months, we should see more of those vehicles running in more cities and these are open to public.

And where we're going from here is that we're building a new vehicle called Cybercab from the ground up. Those are two-seater vehicles with no steering wheel, no accelerator pedal, and no brake pedal. So the vehicle is built specifically just for full self driving. It has a very minimum cost per mile. So you'll be able to actually use this vehicle to do travel that's cheaper than you take a public transportation today.

Going back to the Tesla vehicles on the road, we have a few million vehicles on the road. And they have collectively driven just past 10 billion miles on FSD this month. We started to roll out FSD roughly in 2021, and the progress has been very slow. And after we started to ship V12, you can see the miles on FSD essentially improved exponentially. The thing we figure out is that our entire approach before V12 was just wrong. So we were working on a very modular approach like the rest of the industry prior to V12, where we have teams work on perception, localization, navigation, and then predict what the ego vehicle should do and what other vehicles will do, and then plan the path to do that. And then we realized this is actually the wrong approach.

As a human, when you do driving, you make a lot of decisions implicitly. You are not thinking about a lot of the modular output and then try to chain them together. What you want to do is just drive like a five-star Uber driver. So this is where we basically pivoted to our new end-to-end approach, where we just learn how human drive and use that to train large neural networks to drive the car. And the customers seem to love it according to the metrics.

The case for end-to-end driving

- Codifying human values is incredibly difficult
- Interface between perception, prediction and planning is ill-defined
- Easily scalable to handle the fat and long tail of real-world robotics
- Homogenous compute with deterministic latency
- Overall, on the correct side of scaling w.r.t the bitter lesson

We used to spend a lot of time writing code on what human makes a decision while driving. And it turns out that it works pretty decent in nominal scenarios, but driving is full of edge cases and long-tail events where in the limit it's extremely hard to write.

In the old modular days, I personally worked on a lot of perception tasks where I had to do a lot of detection and occupancy, and also the detection had to include the uncertainty prediction so that prediction and planning take that into account. And also we found out that the better perception model does not necessarily mean a better final outcome. This approach is actually not scalable at all. And especially once you get to the fat and long-tail events of real-world scenarios, just directly predicting what you're supposed to do is a much better approach. We also think this enables us to use homogeneous compute with very deterministic latencies, because in the Tesla vehicle today, we don't really have a different modular software running in a way where computers are separated to run different modules. There's just one large neural network, and we have photons coming in at 36 FPS, and then we pass through all the information through the network and make a decision on what to do immediately. So you can tell that the latencies are extremely deterministic. No matter the scenario, it's easy or complicated. It runs at the same cadence. It's very good for compute optimization.

Overall, we think this approach is really on the correct side of scaling with respect to the bitter lesson. Instead of trying to bake in the human knowledge, what we do is just scale up the model, scale up the data, and try to get a better end product.

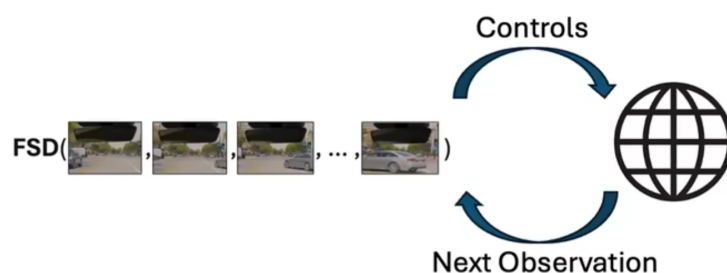
At the core, we need a "**simple**" function

Controls = **FSD**( ,  ,  , ... )

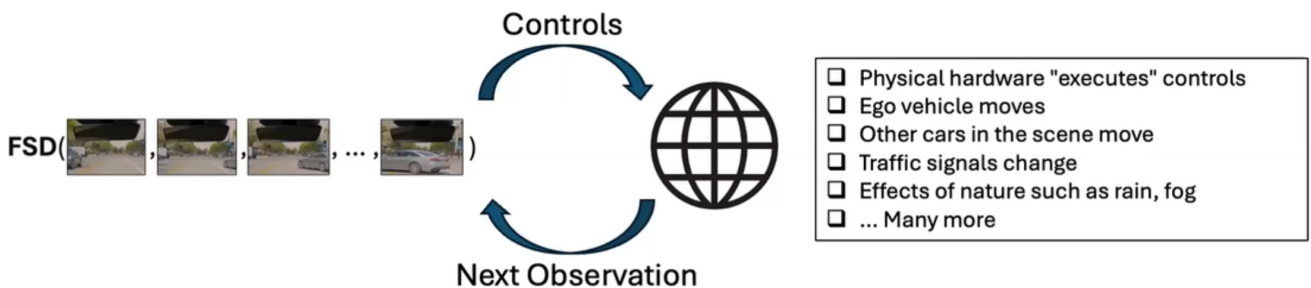
Given **observations**, predict the next **controls** for the car

At the core, we really just need a very simple function. We take all the videos as inputs, and we call a simple function to decide what to do. That's literally what it does. Here, it's video in and action out. And we just give all the observations from the surrounding environment. We just predict what the next control we should perform for the car. And this control obviously will act in the real world, and the real world will change accordingly and will give you the next observations. And we just run this at an extremely high frame rate to solve driving directly. And what is different from this and some of the training recipes of the LLM is that in real world, the world is very messy.

The function then acts in the real-world



The environment dynamics in real-world are far more complex than digital world



As you make a prediction of what to do, the world is also changing around you. So first of all, the model from vision as input to control has a non-zero latency. Basically, you're making a decision and you're acting on the decision, and it will take some delay in order for that to happen. And in the limit, every millisecond really matters.

Second is that the world is also evolving around you. So there's a lot of uncertainties in the other participants in the scene. Other cars might do things that is unpredictable, or it might even be some adversarial cases where things are happening out of the ordinary. And we have to take that all into account and bake in the model. And the nature changes quite a bit. When you're trying to solve self-driving at a scale, that means that the vehicle should be able to go to any environment and perform the task. And driving in the real world is incredibly diverse.

The most natural API is video input and action output. All other tasks are proxy tasks. Sometimes you run into this extremely complicated scenario where some adversarial actor in the scenes do something crazy, and you have to react to it.

Video in, Action out

Tesla has a very, very large-scale dataset of (video, action) pairs:

- ~500 years of drive time every day

Some problems though...

- The input space is very large, and the action space is very small. Massively underconstrained.
- Input space: 36 Hz high resolution input across 8 cameras around the vehicle.
- Output space: mostly 2 degrees of freedom, steering angle and acceleration.
- Even for short durations we would have $> 2,000\times$ more tokens than a $224 \times 224 \times 3$ RGB image processed with CLIP

You obviously need to train on a very large amount of data. We have about seven million cars on the road. They are essentially collecting data for us, and it covers all kinds of different scenarios. And every single day, the entire Tesla fleet drives about 500 years of driving time. On the input level, you can imagine this input space is extremely large. So we have 36 Hz high-resolution input across eight cameras as input. So if you just naively count those pixels and do the normal ways, you patchify it and you tokenize it, we're talking about billions and billions of tokens. Obviously, that won't work at all. So we have to figure out extremely complicated ways to actually compress the input video to a manageable context length in order to tokenize the input. So in an output space, we have an inverse of the problem. Most of the driving is actually just two degrees of freedom. So now we're talking about essentially very, very large input space comes in, and it outputs just two numbers. If you're used to training large models, you will understand that training this naively is very, very unstable.

To give you some example here, I have four different videos here where it's very easy to understand on the input level. Those are four completely different environments. But the action is more or less the same, just drive straight. So if you just train this naively, the model will be very hard to learn to understand the environment. Sometimes the model will just say, OK, if I just drive straight, I'll get an answer right for most of the cases, which obviously the model is not good.

To make matters worse, even with huge datasets, how do you learn the correct causation from millions of pixels per frame? If you look at this frame, there's a lot of things going on. For a neural network, it is trying to parse all the information.

It turns out that when we were working on modular approaches, even though it didn't scale very well, but we still made a lot of explicit tasks that are coming in very handy in terms of helping the model to train efficiently. So essentially, they become these proxy tasks that we use as co-training tasks. When we train the end-to-end model, we also predict all these different tasks simultaneously. Then they really help the gradient flow through the network. And we're able to learn the causation of all the driving decisions from the video much, much more precisely.

What are the additional "proxy" tasks that can help training efficiency?

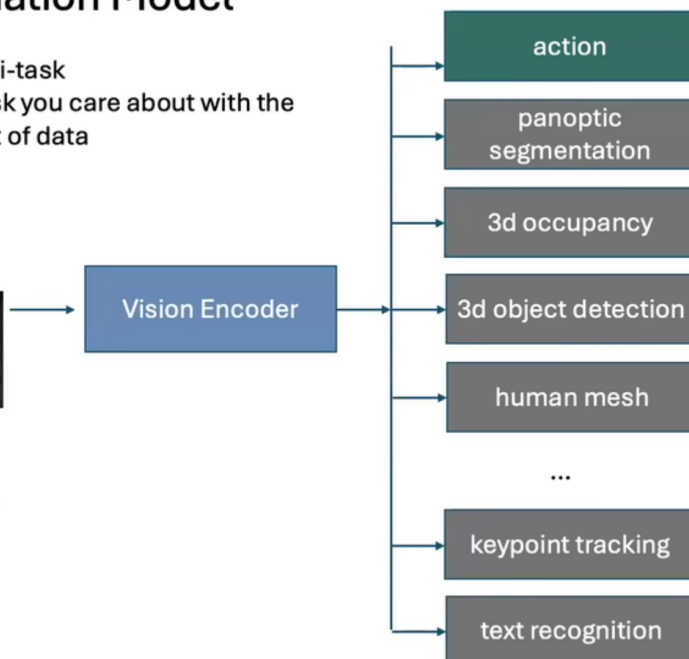
3D geometry	OCR and sign understanding
2D and 3D object detection	Image classification
Object tracking	Scene understanding
Semantic segmentation	Visual Question Answering
Instance segmentation	Localization and mapping
Dense depth	Future video frame prediction
Scene and optical flow	Image reconstruction

Video Foundation Model

- Massively multi-task
- Train on the task you care about with the largest amount of data



36 Hz high-res video
from 8 camera streams



We train this large video foundation model: we take all this input and run the vision encoder, even though at a test time, when we do the action as how do we drive. But at the training time, we predict all the tasks, including a lot of segmentation tasks, geometry, human mesh, and keypoint tracking to actually help the model understand what's going on.

We're also adding internet data to try to make the model reason and generalize better. So we have a lot of data learned from the fleet, but sometimes certain events, you don't really want to just wait until it happens. A lot of stuff is already included in the internet. It turns out that you can train the system on all those tasks, and it works really well. We're even including data from indoor environments, from our Optimus lab, to understand a lot of indoor objects. To give you an example, if you see a chair on a highway that you have to avoid, you shouldn't need to see a lot of chairs on a highway to understand that's a chair. An internet video or some internal, some inside, indoor data should just easily capture a chair, and you'll be able to understand this very well.

The "Secret" Recipe

More Data, Bigger Models = Better Generalization

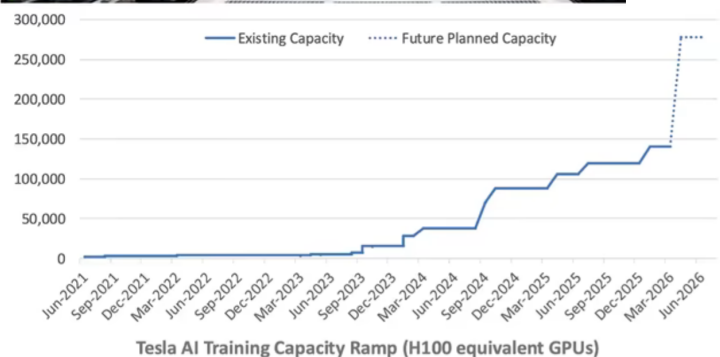
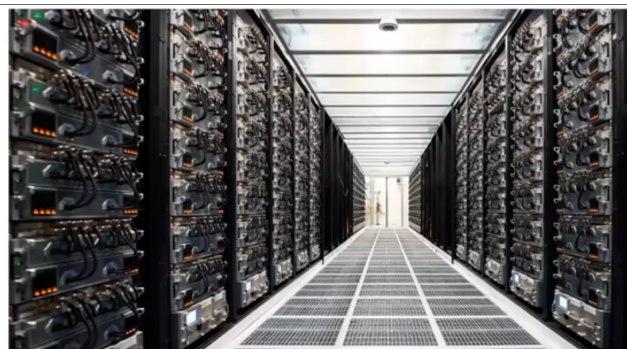
The secret recipe we think is just that you put more data on a bigger model, and then we train with a large amount of compute, and we'll achieve better generalization. So on the data side, we have a very, very large data set. A typical data set that any engineer trains and tests on a daily basis has over 120 years of driving and hundreds of petabytes of data. So that's extremely very large, and we have to invent a lot of tuning internally to handle such a high I/O, because there's no open source library that's capable of doing this.

But data quantity is not the only important thing when we talk about data. We also need very high-quality data and highly diverse data.

When we talk about self-driving, a lot of people like to talk about edge cases, but it turns out when you have seven million cars on the road, any edge cases happens on a daily basis. Those are just some random events we found from our customer. Essentially, if you drive as a single human, you probably won't see this for your entire lifetime. And people usually see this and think this is incredibly crazy, and they will have to use simulation to simulate all this. But if you actually have a very good strong data engine running on a very large fleet, you get all this data every single day. And then we put this data to use in our incredibly large GPU supercomputing clusters.

Compute

- Large GPU supercomputing clusters
- Small engineering team
- High GPU-per-engineer resource
- Getting even more compute!



We're approaching 300,000 H100 equivalent in our compute. And to make it even better, we have a very small engineering team. And what we believe is that we just need a very small and strong engineering team, and we have tons of compute. So as a result, we probably have the highest GPU per engineer resource in the entire industry, and everybody is empowered with

thousands and thousands of GPUs. And it's a very good way to empower the developers and to make the product better.

The two things are really important here: First is the Internet knowledge. Second is reasoning.

You want to solve the problem by not just leveraging driving data, because human learn through other means as well before you can drive. Internet-data pretraining and reasoning, so that the car is able to solve complicated scenarios and unseen scenarios is important. And we definitely leverage all this capability to make the generalization much better for our models.

For all the 3D tasks we have, we've been working on this for quite some time now. So we have all the models to actually run it. Basically, we use a teacher model to train the camera model.

We're baking the collision check into training through reinforcement learning. Essentially, the collision check serves as a verifiable reward. So we know that all collisions are bad. It's actually very straightforward to bake that in at training time. And at test time, we actually mostly rely on the model to do that.

If your data set is small, the curation doesn't really make much sense. So basically, quantity is more important than the rest of the stuff, but when you have a data set at our quantity, we do spend a lot of time on the data set curation. Let's say we have millions of cars driving on the road. If I randomly sample the data back, 90% of the data will be driving straight on the highway. So they're not useful at all. So you have to come up with all the tricks to actually figure out the things that are actually interesting. And also things that maybe the model did, the human driver disagreed with, and you can get all this data back and train the model. Those are much more data efficient.

First, you have to use your intuition. If you find a model fails at doing something, you should be able to trace back to understand what is actually suboptimal. It's a lot of trial and error to debug such models where ultimately what you have to think about is that those tasks are helping the gradient flow of the network. The accuracy of those tasks are not the final product you're looking for, what you're looking for is just the driving. You don't want to just look at all the proxy tasks and say I want to achieve SOTA on every one of these metrics. They don't necessarily help you get the final model to the better state. Sometimes you just need those tasks to be healthy enough in order to get the benefit of that. So ultimately, you have to measure the tasks that you actually care, you actually deploy in the real world, which in our case is driving actions.